Propuesta solución desafio I

1st jesús Sebastián Mendoza
dept. Ingenieria Electrónica
Universidad de Antioquia
Medellín, Antioquia
jsebastian.mendoza@udea.edu.co

2nd Danna Isabela ochoa
dept. Ingenieria Electrónica
Universidad de Antioquia
Medellín, Antioquia
danna.ochoa@udea.edu.co

I. Introduction

En este documento se despliega una breve explicación que abarca una posible solución al desafio uno.

A. Objetivos

• Desarrollar la capacidad de solución de problemas en los estudiantes, enfrentándolos a problemáticas cercanas a situaciones reales.

• Evaluar si el estudiante ha adquirido las destrezas y conocimientos fundamentales de la programación en C++, tales como el uso de estructuras de control (secuenciales, iterativas y condicionales), manejo de tipos de datos, operaciones a nivel de bits, punteros, arreglos, memoria dinámica y funciones.

II. Desarrollo del tablero

A. Estructura de datos del tablero

1) Elección del tipo de dato: Cada celda del tablero representa un estado binario: ocupada (`1`) o libre (`0`). Para minimizar el uso de memoria se empaquetan 8 celdas por byte usando `unsigned char`. De esta forma se evita utilizar un byte completo por celda.

El tipo `unsigned char` se elige porque garantiza exactamente 8 bits útiles (rango 0–255). Al ser `unsigned`, no existe bit de signo, lo que permite que los desplazamientos (`<<`, `>>`) y las máscaras (`&`, `|`, `~`) se comporten de forma predecible durante las operaciones de manipulación de bits.

2) Representación de una celda: Dentro de cada byte, cada bit corresponde a una celda de la fila:

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
1	0	1	0	0	1	0	1
#	.	#	.	.	#	.	#

Un byte con valor `0x00` representa una fila completamente libre, mientras que un byte con valor `0xFF` representa una fila completamente ocupada.

Identify applicable funding agency here. If none, delete this.

3) Bytes por fila y validación de dimensiones: La cantidad de bytes necesarios para representar una fila del tablero depende del número de columnas (`ancho`). Dado que cada byte contiene 8 bits, la cantidad de bytes por fila se calcula como:

$$\texttt{bytes_por_fila} = \frac{\texttt{ancho}}{8}$$

Para que esta representación sea válida, el ancho del tablero debe ser múltiplo de 8. En caso de que el valor ingresado no cumpla esta condición, se mostrara por pantalla un aviso para que sea corregido.

4) Jerarquía de memoria: El tablero se representa como un doble puntero `unsigned char**`. Esto corresponde a un arreglo dinamico de punteros donde cada elemento apunta a una fila del tablero almacenada en memoria dinámica.

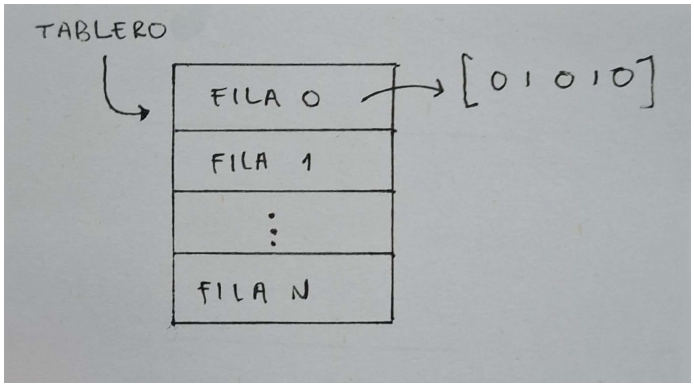

Fig. 1. Representación del tablero en memoria.

La memoria utilizada para almacenar los datos del tablero es:

$$\texttt{alto} \times \frac{\texttt{ancho}}{8} \quad \text{bytes}$$

5) Reserva e inicialización de memoria: Primero se reserva memoria para el arreglo de punteros que representa las filas del tablero. Posteriormente se reserva memoria para cada fila de forma individual.

Cada fila contiene `bytes_por_fila` bytes. Durante este proceso la memoria se inicializa en cero, lo que garantiza que

todas las celdas del tablero comiencen en estado libre. Esto significa que todos los bits inicialmente tienen valor 0.

6) Liberación de memoria: Debido a que el tablero utiliza memoria dinámica, es necesario liberar correctamente todos los bloques reservados una vez que dejan de ser necesarios.

La liberación se realiza en orden inverso al de la reserva: primero se libera la memoria correspondiente a cada fila del tablero y posteriormente se libera el arreglo de punteros que contiene las direcciones de dichas filas. De esta manera se evita la aparición de fugas de memoria.

B. Acceso a celdas del tablero

Para acceder a una celda específica del tablero es necesario determinar qué byte la contiene y cuál es el bit correspondiente dentro de ese byte.

Si una celda se encuentra en la fila f y columna c, entonces:

$$\text{byte} = \frac{c}{8}$$

$$\text{bit} = c \bmod 8$$

El valor del bit puede obtenerse desplazando el byte hacia la derecha hasta que el bit deseado se encuentre en la posición menos significativa, y posteriormente aplicando una máscara con el operador AND.

Para marcar una celda como ocupada se utiliza una máscara con un único bit en la posición correspondiente y se aplica una operación OR. De forma similar, para liberar una celda se utiliza una máscara invertida junto con el operador AND, lo que permite modificar únicamente el bit deseado sin afectar los demás bits del byte.

III. Generación de los tetrimos y rotaciones

Para poder generar las diferentes figuras de los tetrimos, primero creamos las formas bases de cada tetrimo mediante asignaciones directas de la siguiente forma:

```
generarFormaBase(tipo, forma[4]):
    si tipo == 0 (I):
        forma[0] = 0b11110000
    sino si tipo == 1 (O):
        forma[0] = 0b11000000
        forma[1] = 0b11000000
    sino si tipo == 2 (T):
        forma[0] = 0b11100000
        forma[1] = 0b01000000
    sino si tipo == 3 (S):
        forma[0] = 0b01100000
        forma[1] = 0b11000000
    sino si tipo == 4 (Z):
        forma[0] = 0b11000000
        forma[1] = 0b01100000
    sino si tipo == 5 (J):
        forma[0] = 0b10000000
        forma[1] = 00b11100000
    sino si tipo == 6 (L):
        forma[0] = 0b00100000
        forma[1] = 0b11100000
```

Fig. 2. Figuras

Y a partir de estas formas bases puedo generar las rotaciones a la izquierda o a la derecha:

Rotación 90° a la derecha

```
rotarDerecha(forma[4], nuevaForma[4]):

    nuevaForma[0] = ((forma[3] & 0x80) >> 0) | ((forma[2] & 0x80) >> 1)
                  | ((forma[1] & 0x80) >> 2) | ((forma[0] & 0x80) >> 3)

    nuevaForma[1] = ((forma[3] & 0x40) << 1) | ((forma[2] & 0x40) >> 0)
                  | ((forma[1] & 0x40) >> 1) | ((forma[0] & 0x40) >> 2)

    nuevaForma[2] = ((forma[3] & 0x20) << 2) | ((forma[2] & 0x20) << 1)
                  | ((forma[1] & 0x20) >> 0) | ((forma[0] & 0x20) >> 1)

    nuevaForma[3] = ((forma[3] & 0x10) << 3) | ((forma[2] & 0x10) << 2)
                  | ((forma[1] & 0x10) << 1) | ((forma[0] & 0x10) >> 0)
```

Fig. 3. Rotación 90ª derecha

El objetivo de este procedimiento es generar una nueva orientación de la pieza rotada 90° hacia la derecha. La pieza está representada por el arreglo `forma[4]`, donde cada elemento corresponde a una fila de la pieza codificada en un *byte*, utilizando únicamente los cuatro bits más significativos. El resultado de la rotación se almacena en `nuevaForma[4]`.

La rotación se basa en la siguiente idea: cada columna de la pieza original se convierte en una fila de la pieza rotada, leyendo los bloques desde la fila inferior hacia la superior. Esto reproduce el comportamiento de una rotación de 90° en sentido horario.

Para construir cada fila de `nuevaForma`, se extraen bits específicos de las filas de `forma` utilizando máscaras binarias (`0x80`, `0x40`, `0x20`, `0x10`). Estas máscaras permiten aislar una columna específica de la pieza mediante la operación AND (`&`). Posteriormente, los bits obtenidos se desplazan hacia la izquierda o derecha para colocarlos en la posición correcta dentro de la nueva fila.

Por ejemplo:

- `forma[3] & 0x80` extrae el bit de la columna más a la izquierda de la fila inferior de la pieza.
- `forma[2] & 0x80`, `forma[1] & 0x80` y `forma[0] & 0x80` extraen los bits de esa misma columna en las filas superiores.

Cada uno de estos bits se desplaza con los operadores << o >> para alinearlo correctamente dentro de `nuevaForma[0]`, y luego todos se combinan mediante la operación OR (`|`), que permite unir los bits en un solo *byte*.

Rotación 90° a la izquierda

```
rotarIzquierda(forma[4], nuevaForma[4]):

    nuevaForma[0] = ((forma[0] & 0x10) << 3) | ((forma[1] & 0x10) << 2)
                  | ((forma[2] & 0x10) << 1) | ((forma[3] & 0x10) >> 0)

    nuevaForma[1] = ((forma[0] & 0x20) << 2) | ((forma[1] & 0x20) << 1)
                  | ((forma[2] & 0x20) >> 0) | ((forma[3] & 0x20) >> 1)

    nuevaForma[2] = ((forma[0] & 0x40) << 1) | ((forma[1] & 0x40) >> 0)
                  | ((forma[2] & 0x40) >> 1) | ((forma[3] & 0x40) >> 2)

    nuevaForma[3] = ((forma[0] & 0x80) >> 0) | ((forma[1] & 0x80) >> 1)
                  | ((forma[2] & 0x80) >> 2) | ((forma[3] & 0x80) >> 3)
```

Fig. 4. Rotación 90ª a la izquierda

La rotación hacia la izquierda sigue el mismo principio utilizado en la rotación hacia la derecha: reorganizar los bits de la pieza para generar una nueva orientación. La pieza original se encuentra en el arreglo `forma[4]` y el resultado de la rotación se almacena en `nuevaForma[4]`.

En este caso, la lógica consiste en que cada columna de la pieza original se transforma en una fila de la pieza rotada, pero ahora leyendo los bloques desde la fila superior hacia la inferior, lo cual corresponde a una rotación de 90° en sentido antihorario.

Para lograr esto, el algoritmo utiliza máscaras binarias (`0x10`, `0x20`, `0x40`, `0x80`) que permiten aislar una columna específica de la pieza mediante la operación AND (`&`). Por ejemplo:

- `forma[0] & 0x10`, `forma[1] & 0x10`, `forma[2] & 0x10` y `forma[3] & 0x10` extraen los bits de la columna más a la derecha de la pieza original.

Una vez extraídos, estos bits se desplazan con los operadores << o >> para ubicarlos en la posición correcta dentro de la nueva fila.

IV. Detección de Colisiones

Una colisión ocurre cuando algún bit activo de la pieza coincide con:

1. Un bit activo en el tablero (colisión con pieza fija)
2. El borde izquierdo del tablero (colisión con pared izquierda)
3. El borde derecho del tablero (colisión con pared derecha)
4. El fondo del tablero (colisión con suelo)

La pieza vive en su propio sistema de 4 bits, pero el tablero tiene n bytes. Antes de comparar, necesitamos proyectar la fila de la pieza sobre la fila del tablero en la posición correcta.

La posición horizontal de la pieza se guarda en col (columna del tablero donde empieza el bit 7 de la pieza). Necesitamos saber:

byteInicio = col / 8 (en qué byte del tablero empieza)

bitOffset = col modulo 8 (cuántos bits desplazar dentro del byte)

Para proyectar una fila de la pieza sobre el tablero:

(8 bits, solo 4 relevantes: los altos)
```
filaPieza = forma[i]
```

(bits que caen en byteInicio)
```
parteAlta = filaPieza >> bitOffset
```

(bits que desbordan al siguiente byte)
```
parteBaja = filaPieza << (8 - bitOffset)
```

Esto es importante porque es posible que la pieza caida en la mitad entre dos bytes entonces hay que saber por ejemplo que bits quedan en el byte 0 y cuales caen el bit1. Ejemplo:

byte0 = 00000011
byte1 = 10000000

Así la pieza queda alineada con el tablero para poder comparar.

Tipos de colisión con operaciones de bits:

Colisión con piezas fijas en el tablero

El objetivo de este algoritmo es determinar si una pieza del juego intenta ocupar una posición del tablero que ya está ocupada por otro bloque. Para ello se comparan los bits activos de la pieza con los bits activos del tablero en la posición donde se desea colocar la pieza.

La pieza se representa mediante un arreglo forma[4], donde cada elemento corresponde a una fila de la pieza codificada en un byte. En cada byte únicamente se utilizan los cuatro bits más significativos, que indican la presencia o ausencia de bloques.

El tablero se almacena como una matriz de bytes tablero[fila][byte], donde cada bit representa una celda ocupada o vacía.

Para cada fila i de la pieza (0 a 3):

(fila real en el tablero)
```
filaTablero = fila + i
```

```
parteAlta = forma[i] >> bitOffset
parteBaja = forma[i] << (8 - bitOffset)
```

si `tablero[filaTablero][byteInicio] & parteAlta != 0:`
  //hay colision

si `bytesporFila > byteInicio + 1:`
  si `tablero[filaTablero][byteInicio + 1] & parteBaja != 0:`
    //hay colision

Primero se recorren las filas de la pieza Luego se calcula la fila correspondiente en el tablero Por lo tanto, filaTablero indica la fila real del tablero donde se proyecta esa fila de la pieza

Para ubicar la pieza en la columna correcta del tablero se realizan desplazamientos de bits

Para detectar colisiones se utiliza la operación lógica AND entre los bits del tablero y los bits de la pieza

Aquí se comparan:

tablero[filaTablero][byteInicio] (byte del tablero en la posición donde se intenta colocar la pieza.)

parteAlta (bits de la pieza alineados en ese mismo byte.)

La operación AND compara bit a bit ambas variables. El resultado será distinto de cero únicamente si existe al menos un bit en el que tanto la pieza como el tablero tienen un valor de 1, lo que indica que ambos intentan ocupar la misma celda

Colsiones con el suelo

El objetivo de este procedimiento es verificar si algún bloque de la pieza intenta salir por la parte inferior del tablero. La pieza se representa mediante el arreglo `forma[4]`, donde cada posición corresponde a una fila de la pieza. El índice `i` (de 0 a 3) permite recorrer dichas filas.

Para cada fila `i` de la pieza se realiza la siguiente verificación:

si `forma[i] != 0:`
  si `fila + i >= alto:`
    //hay colision

Primero se comprueba `forma[i] != 0`. Esto significa que esa fila de la pieza contiene al menos un bloque activo. Si la fila no tiene bloques, no es necesario evaluarla. Luego se calcula la expresión `fila + i`, donde:

- `fila`: es la fila del tablero donde comienza la pieza.
- `i`: es la fila relativa dentro de la pieza.

La suma `fila + i` indica la fila real del tablero donde se ubicaría esa parte de la pieza. Finalmente, se compara este valor con `alto`, que corresponde al número total de filas del tablero. Si se cumple que `fila + i >= alto`, significa que algún bloque intentaría ubicarse por debajo de la última fila válida, lo cual indica una colisión con el suelo.

Colisión con los bordes del tablero

El objetivo de estas verificaciones es asegurar que la pieza no salga del tablero por los lados cuando se intenta mover horizontalmente.

Colisión con el borde izquierdo:

Para cada fila `i` **de la pieza:**
  si `forma[i] != 0`:
    si `col < 0`:
      hay colision

Se recorren las filas de la pieza (`i` de 0 a 3). La condición `forma[i] != 0` indica que esa fila contiene bloques, por lo que debe verificarse su posición. Luego se evalúa `col < 0`, donde `col` representa la columna del tablero donde comienza la pieza. Si es menor que cero, la pieza intentaría ubicarse fuera del tablero por la izquierda.

Colisión con el borde derecho:

Para cada fila `i` **de la pieza:**
  `columnaDerechaPieza = col + anchoPieza(forma)`
  si `columnaDerechaPieza > ancho`:
    hay colision

Primero se calcula `columnaDerechaPieza`, que corresponde a la columna más a la derecha que ocuparía la pieza. Este valor se obtiene sumando `col` con `anchoPieza(forma)`, función que devuelve el ancho de la pieza en columnas. Luego se compara con `ancho`, el número total de columnas del tablero; si la pieza lo supera, se detecta una colisión.

V. Fijación de Piezas y Limpieza de Filas

Cuando hayColision detecta que la pieza no puede bajar más, sus bits deben escribirse permanentemente en el tablero usando OR, exactamente igual que la proyección que ya calculamos en colisiones.

Operación de fijación con OR La lógica es directa: los bits activos de la pieza se suman al tablero sin borrar lo existente:

Primero se calcula la posición horizontal de la pieza dentro del arreglo de bytes del tablero:

```
byteInicio = col / 8
bitOffset = col modulo 8
```

Luego se recorren las filas de la pieza (**Para** `i` **desde 0 hasta 3**). Cada iteración corresponde a una fila de `forma[i]`.

```
hayColision(tablero, forma[], fila, col, ancho, alto, bytesporFila):

    byteInicio = col / 8
    bitOffset  = col % 8

    Para i desde 0 hasta 3:

        si forma[i] == 0: continuar    ← fila vacía, saltar

        — Colisión con suelo —
        si fila + i >= alto:
            retornar verdadero

        — Colisión con borde izquierdo —
        si col < 0:
            retornar verdadero

        — Colisión con borde derecho —
        si col + anchoPieza(forma) > ancho:
            retornar verdadero

        — Proyectar fila de pieza sobre tablero —
        parteAlta = forma[i] >> bitOffset
        parteBaja = forma[i] << (8 - bitOffset)

        — Colisión con piezas fijas —
        si tablero[fila+i][byteInicio] & parteAlta != 0:
            retornar verdadero

        si byteInicio + 1 < bytesporFila:
            si tablero[fila+i][byteInicio+1] & parteBaja != 0:
                retornar verdadero

    retornar falso    ← ninguna colisión detectada
```

Fig. 5. Función general de verificación

```
[Mover izquierda]
    si hayColision(tablero, forma, fila, col-1, ...):
        ignorar movimiento
    sino:
        col = col - 1

[Mover derecha]
    si hayColision(tablero, forma, fila, col+1, ...):
        ignorar movimiento
    sino:
        col = col + 1

[Bajar]
    si hayColision(tablero, forma, fila+1, ...):
        fijarPieza()        ← la pieza toca fondo o pieza fija
    sino:
        fila = fila + 1

[Rotar]
    generar formaRotada
    si hayColision(tablero, formaRotada, fila, col, ...):
        descartar rotación
    sino:
        aplicar rotación
```

Fig. 6. Uso de hayColision en el juego

(fila de la pieza sin bloques, se omite)
si `forma[i] == 0: continuar`

Cuando la fila tiene bloques, estos se proyectan horizontalmente en el tablero utilizando desplazamientos de bits para alinearlos con la cuadrícula. Finalmente, los bits de la pieza se escriben en el tablero utilizando el operador `OR` inclusivo:

(escribir bloques en el primer byte del tablero)
`tablero[fila+i][byteInicio] |= parteAlta`

Esta operación combina los bits de `parteAlta` con el byte correspondiente del tablero. Si en el tablero ya existían bloques, estos se mantienen, mientras que los nuevos bloques de la pieza se agregan sin borrar lo anterior. Luego se verifica la siguiente condición:

(comprobar si existe un segundo byte válido)

```
fijarPieza(tablero, forma[], fila, col, bytesporFila):
    byteInicio = col / 8
    bitOffset  = col % 8

    Para i desde 0 hasta 3:

        si forma[i] == 0: continuar    fila vacía, saltar

         Proyectar fila de pieza
        parteAlta = forma[i] >> bitOffset
        parteBaja = forma[i] << (8 - bitOffset)

         Escribir en tablero con OR
        tablero[fila+i][byteInicio] |= parteAlta

        si byteInicio + 1 < bytesporFila:
            tablero[fila+i][byteInicio+1] |= parteBaja
```

Fig. 7. Operación de fijación con OR

si `byteInicio + 1 < bytesporFila:`
`tablero[fila+i][byteInicio + 1] |= parteBaja`

Esto comprueba que la pieza no se desborde fuera del ancho de memoria del tablero. Si la pieza ocupa también ese espacio, se escriben los bits restantes almacenados en `parteBaja` mediante la misma operación lógica

Limpieza de filas

Una fila está llena cuando todos sus bits activos suman el ancho del tablero. Con bytes, esto significa que cada byte de esa fila debe ser 0xFF (todos los bits en 1)

La máscara 0xFF verifica los 8 bits del byte de una sola operación, sin recorrer bit por bit.

```
filaLlena(tablero[fila], bytesporFila):

    Para j desde 0 hasta bytesporFila-1:
        si tablero[fila][j] & 0xFF != 0xFF:   // AND con máscara completa
            retornar falso

    retornar verdadero
```

Fig. 8. detectar fila llena

Eliminación y descenso de filas — gestión de memoria

Cuando se detecta que una fila esta llena, manipulamos directamente los punteros de las filas, de esa forma podemos ir actualizando el tablero con las nuevas filas disponibles.

1. Identificar qué filas están llenas (Punteros)
2. Limpiar esas filas con 0x00
3. Colocarlas al tope del tablero, es decir, ubicar esos punteros en la parte alta del tablero.
4. Los punteros no llenos no se toca y esos son los que van quedando mas abajo.

VI. Game Over y Generación de Nueva Pieza

El Game Over ocurre cuando una pieza nueva no puede colocarse en la posición inicial del tablero. Esto significa que la zona superior del tablero ya tiene bits activos que colisionan con la pieza entrante.

La posición inicial siempre será el centro del tablero. La detección es simplemente reutilizar hayColision que ya tenemos. No necesitamos revisar todo el tablero, solo las filas superiores donde aparece la pieza. Como toda pieza cabe en 4 filas, solo las filas 0 a 3 son relevantes.

La pieza nueva ocupa máximo las filas 0, 1, 2, 3 Si cualquiera de esas filas tiene bits que colisionan esto genera GAME OVER. Esto ya lo maneja hayColision internamente

Para la generación de la figura de forma aleatoria se debe de generar algún mecanismo que genere numeros de forma aleatoria y para ello se nos ocurre utilizar el valor de alguna variable que esté cambiando constantemente durante el juego. Tambien se podria utilizar alguna semilla y que mediante un ciclo este esté cambiando constantemente con el flujo del juego.

VII. Flujo general del juego

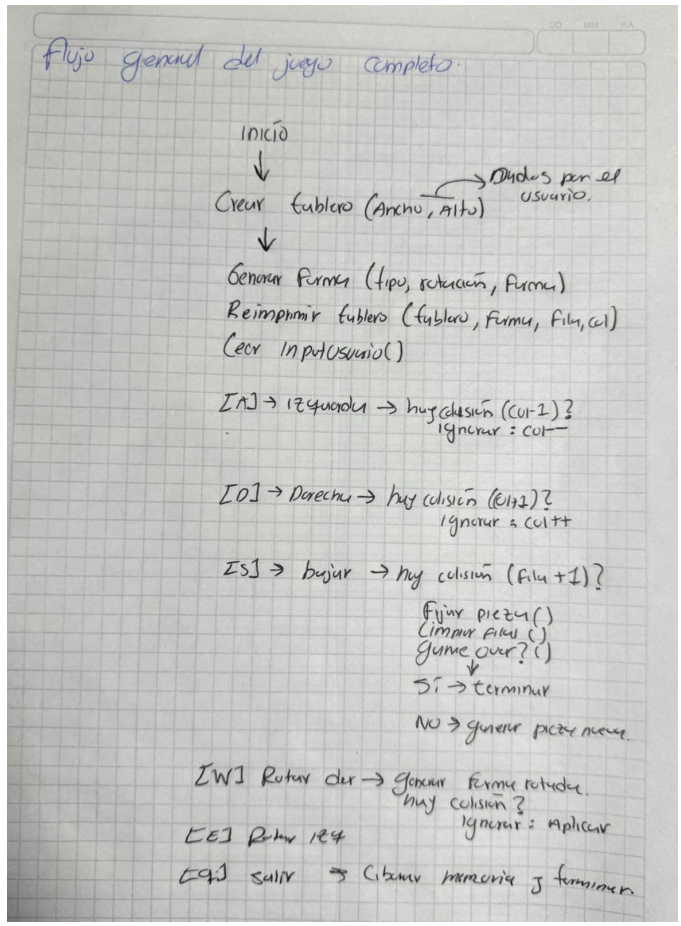

Fig. 9. Flujo general del juego

Referencias

- Diapositivas de clase – Profesor Aníbal José Guerra Soler, curso Informática II. Disponible en: Material de clase Informática II